QShield-X: A Formally Checked, FPGA-Targeted Union-Find Decoder for the Surface Code, with a WebAssembly Runtime and a V8 Code-Generation Fix

QShield-X project

Draft — all numbers are generated from `results/` by `bench/claims.py`

Abstract

We ask whether hardware–software co-design with formally checked execution protocols gives a better logical-error / latency / resource trade-off than a fixed software decoder for rotated surface-code memory experiments under circuit-level depolarizing noise. We build *QSX-UF*, a union-find decoder whose semantics are evaluation-order independent, so that a fully parallel SystemVerilog implementation and a sequential C++ golden model agree bit for bit, including the correction bitmap and the cycle count. They agree on 3,840,000 plus 1,040,000 co-simulated syndromes, with 0 mismatches. k-induction proves the shell's protocol properties for every core behaviour and the core's safety and watchdog properties at $d = 3, 5, 7$. TLA+ model checking of the host/accelerator/fallback protocol found a liveness bug, which we fixed. On a Lattice LFE5U-85F (open toolchain, after place-and-route, no board), the decoder is *not* faster than PyMatching on one CPU core in p99 latency: 833 ns vs. 0.54 µs at $d = 3$ [sim cycles / PnR Fmax vs. CPU]. Its worst case, however, is 5.0× lower than the CPU's observed tail and is guaranteed by proof. Frames over budget come back as `WATCHDOG` and go to the verified software path. QSX-UF costs 1.28× PyMatching's logical error rate at $d = 7$, and fully parallel cores exceed the device beyond $d = 5$. The answer is therefore *mixed*: co-design buys a verifiable deadline, not raw speed, on this device class. We also contribute an exact cycle accounting that identifies the architecture's latency lever, an upstreamable V8 arm64 code-generation fix found while profiling the decoder's WebAssembly build, adaptive and learned decoding under noise shifts, and a federated-learning simulation with differential privacy that locates a privacy–utility knee.

1 Introduction

Fault-tolerant quantum computing needs a classical decoder that keeps up with the syndrome stream. For superconducting devices one round of stabilizer measurements takes on the order of a microsecond, so a decoder that falls behind builds up a backlog that eventually stalls the logical computation [9, 11]. Minimum-weight perfect matching (MWPM) is accurate and fast in software [13, 14], but its data structures (priority queues, alternating trees) are awkward to bound in hardware. Union-find (UF) decoding [8] uses small integer state and local operations and is the basis of several hardware decoders [6, 19, 2].

A decoder in a control stack is also a *protocol participant*: it receives frames under backpressure, returns results tagged with the measurement round they belong to, may time out, may be reconfigured, and may be bypassed by a software fallback. These behaviours are where integration bugs live, and none of them is captured by a logical-error-rate plot. QShield-X treats them as first-class, checkable properties.

Question. Can hardware–software co-design with formally checked execution protocols deliver a better logical-error / latency / resource trade-off than a fixed software-only decoder, for rotated surface-code memory experiments with $d \in \{3, 5, 7, 9\}$ under uniform circuit-level depolarizing noise?

Contributions.

- **QSX-UF**, a Jacobi-synchronous union-find decoder with a canonical spanning forest, defined so that parallel RTL and sequential C++ are bit- and cycle-exact (§3); weighted (3-bit) and unweighted variants; P_L measured against PyMatching with exact binomial confidence intervals (§7.1).

- A parameterised SystemVerilog microarchitecture (AXI-Stream shell, commit-on-last frame FIFO, atomic configuration, watchdog, LANES-wide scans) with Verilator co-simulation and an open-source ECP5 flow (§4, §7.3).
- Formal verification at three levels with stated bounds: SVA/SymbiYosys k-induction on the RTL, TLA+/TLC on the host protocol (a real counterexample and its fix), and Z3 step equivalence of the core (§5).
- A native/Wasm-scalar/Wasm-SIMD runtime study, and an in-tree V8 patch with unittests and an mjsunit test that removes a redundant compare/set from `v128.any_true`-guarded branches on arm64 (§6).
- Extensions: GNN decoding and an adaptive policy under noise shifts; federated training with DP-SGD and simulated secure aggregation; signed and encrypted experiment artefacts (§7.7).

Every number in this paper is generated from a result file that records the git commit, tool versions, seed and configuration. Hardware numbers carry their evidence stage in brackets: [sim] (Verilator or the cycle-exact model), [synth] (Yosys), [PnR] (nextpnr). No number comes from a physical FPGA board, and none of this is an ASIC result.

2 Background

Surface-code memory. We use Stim's `surface_code:rotated_memory_z` circuits [10] with d rounds and the same probability p for two-qubit depolarization after Cliffords, data depolarization before each round, measurement flips, and reset flips. A shot fails if the decoder's prediction of the logical observable differs from the true flip; $\mathrm{P_L}$ is the failure probability per d-round block. The decoder input is the set of *detection events* (defects); the decoding graph comes from the detector error model with `decompose_errors`, parallel edges merged, and one virtual boundary vertex.

Union-find decoding. UF [8] grows clusters around defects in half-edge steps, merges clusters that touch, stops growing a cluster once it has even parity or touches the boundary, and then *peels* a spanning forest of each cluster to obtain a correction. Weighted UF [15] grows edges at speeds proportional to their log-likelihood weight. The original algorithm grows the smallest odd cluster first; its outcome depends on processing order, which is what QSX-UF removes.

3 QSX-UF: an order-independent union-find

QSX-UF (DECISIONS D1) keeps an integer support $s_e \in [0, t_e]$ per edge, with growth threshold $t_e \in [1, 15]$. An edge is grown iff $s_e = t_e$. Clusters are connected components of grown non-boundary edges. A vertex is *active* iff its cluster has odd parity and no grown boundary edge. One growth round is a synchronous (Jacobi) update $s_e \leftarrow \min(t_e, s_e + a_u + a_v)$ using activities from before the round. Rounds repeat while any vertex is active, up to `max_rounds`; on expiry the status is `TIMEOUT` and the correction is all-zero. The spanning forest is canonical: the root of a cluster is the boundary if the cluster touches it, else its minimum-index vertex; BFS distances are computed in parallel, and each vertex picks the *lowest-index* grown incident edge towards distance -1. Peeling proceeds in decreasing distance. Since every step is a function of the previous state only, a parallel implementation and a sequential one produce identical supports, forests and corrections.

Thresholds are quantised log-likelihood weights, $t_e = \mathrm{clamp}(\mathrm{round}(5\, w_e/\mathrm{median}(w)),\ 1,\ 7)$ (`q5`, 3-bit counters, chosen by a pilot study, DECISIONS D5), or $t_e = 2$ for the minimal-area unweighted variant.

4 Microarchitecture

Core. The edge fabric holds all E support counters and updates them in one cycle per growth round. The vertex fabric holds per-vertex labels and computes cluster labels by min-label propagation over grown edges (balanced min trees), one hop per cycle until convergence. It also holds BFS distances, one-hot parent-edge selection, and self-timed peeling, where a vertex fires once all its children are done. Cluster parity, boundary flags and the activity broadcast are computed by *LANES* scans that process LANES vertices per cycle, so the compute-unit parameter trades area for $2\lceil V/\mathrm{LANES}\rceil$ cycles per round. The FSM is `IDLE`→`LABEL*`→`PAR`→`ACT`→`CHK`→(`GROW`→ $\cdots$)→`DIST*`→`PEH`→`PEEL*`→`DONE`. An optional pipeline register (PIPE) adds one cycle per LABEL and DIST

step to shorten the critical path. Safety mechanisms: the `max_rounds` timeout, a `cycle_budget` watchdog, fail-closed peeling, and zeroed outputs for any non-OK status.

Shell. Input frames are a header carrying a 16-bit round tag plus $\lceil V/32 \rceil$ syndrome beats over AXI-Stream. They enter a BRAM ring of NBUF slots that commits a frame only on its last beat and de-asserts `tready` when full. A dispatcher starts the core only when the configuration is valid and closed. The egress emits a header (magic, status, observable, tag) and optionally the E-bit correction. Malformed frames are answered in order with `FRAME_ERROR`. Configuration is an atomic BEGIN/WRITE/COMMIT session: COMMIT validates only if every threshold word was written and every value is in range; otherwise frames are answered `CFG_INVALID` and never decoded.

5 Verification

5.1 Co-simulation

Verilator models of the core and of the full shell are compared, per syndrome, against the C++ golden model `qsx::HwModel` on status, round count, observable, the full correction bitmap and the cycle count. Inputs are random syndromes over a wide range of defect densities and Stim-sampled syndromes at $p \in \{0.001, 0.003, 0.006, 0.01\}$. The shell tests add input gaps, output backpressure, malformed frames and mid-stream reconfiguration, including deliberately incomplete sessions (Table 2).

5.2 SVA and SymbiYosys

Table 1 lists the properties and their results. The shell harness replaces the core by an abstract stub that may take any latency and return any status and correction, subject only to the core's interface guarantees. Those guarantees are themselves proven on the real core ($d = 3$). The shell proofs therefore hold for every core behaviour. A bounded check of the composition with the real core complements them.

5.3 TLA+ protocol model

`QSXProtocol.tla` models a host that submits rounds to a K-deep accelerator queue, receives tagged responses, filters stale ones, checks status, falls back to software after a deadline or an error, and commits rounds in order. It also models reconfiguration sessions that the accelerator may only accept when empty. The accelerator gets *no* fairness: it may stall or lose requests forever. Safety: NoDoubleCommit, NoSkip, ExclusiveSource (hardware xor software), NoInvalidCommit, NoOverflow. Liveness: every round is eventually committed, under weak fairness of host, timer, fallback and session close. The first version violated liveness: requests that had timed out stayed in a stalled accelerator, filled its queue and blocked every later submission. We fixed it with a submission deadline that bypasses the accelerator. Mutations that remove the stale filter or the status check each violate safety, which shows the invariants are not vacuous.

5.4 Z3 step equivalence

Yosys exports the flattened core as an SMT-LIB transition relation. For *all* register and input values, reachable or not, Z3 [7] proves that one clock step of each FSM phase (LABEL, GROW, DIST, PEH) equals a reference built independently from the graph file. This also validates the table generator. A mutated reference (highest-index parent) is SAT, which guards against vacuity.

6 WebAssembly runtime and a V8 code-generation fix

The same C++ decoder is compiled natively (clang `-O3`, with and without auto-vectorisation/NEON) and with Emscripten to Wasm, both scalar and SIMD128 [12, 16]. All builds produce the same FNV-1a hash over statuses, observables and correction lists on shared test vectors. We measure them in Node.js, in d8 built from source (default tiering, Liftoff only, TurboFan/Turboshaft only) and in headless Chrome.

The SIMD build skips all-zero 16-byte blocks of the bit-packed syndrome with `v128.any_true`. Dumping V8's arm64 code showed that `br_if(v128.any_true(x))` compiles to `umaxp; fmov; cmp; cset; cbnz`:

Table 1: Formal verification jobs (`make formal`). "prove" = k-induction (unbounded); "bmc" = bounded to the stated depth; TLC explores the full state space of the stated model constants.

job	tool	bound	result	s
`ingress.sby:prove_w2_n4`	SymbiYosys	prove depth 24	PASS	2
`ingress.sby:prove_w4_n2`	SymbiYosys	prove depth 24	PASS	2
`top.sby:stub_prove`	SymbiYosys	prove depth 20	PASS	9
`top.sby:real_bmc`	SymbiYosys	bmc depth 40	PASS	181
`top.sby:real_cover`	SymbiYosys	cover depth 60	PASS	1736
`core.sby:prove`	SymbiYosys	prove depth 12	PASS	6
`core.sby:cover`	SymbiYosys	cover depth 48	PASS	2515
`MC_main.cfg`	TLC 2.19	141,744 states, depth 25	no error	15
`MC_main_N5K3.cfg`	TLC 2.19	1,342,628 states, depth 30	no error	157
`MC_mut_stale.cfg`	TLC 2.19	395 states, depth 7	safety violated (expected)	2
`MC_mut_status.cfg`	TLC 2.19	100 states, depth 6	safety violated (expected)	1
`MC_v0_nobypass.cfg`	TLC 2.19	68,025 states	liveness violated (expected)	4
`core_d3_step_equivalence`	Z3	4 steps, 1 transition	all UNSAT	46
`core_d5_step_equivalence`	Z3	4 steps, 1 transition	all UNSAT	17958
`core_d3_step_equivalence_mutant`	Z3	1 steps, 1 transition	mutant SAT (expected)	0
`core.sby:prove_d5`	SymbiYosys	prove depth 12	PASS	51
`core.sby:prove_d7`	SymbiYosys	prove depth 12	PASS	422

Table 2: Co-simulation of the final RTL against the C++ golden model (status, rounds, observable, full correction bitmap and cycle count compared per syndrome). Columns: syndromes / mismatches.

	$d = 3$		$d = 5$		$d = 7$	
core, random syndromes	1,000,000	0	1,000,000	0	1,000,000	0
core, Stim syndromes	100,000	0	100,000	0	100,000	0
core, LANES sweep 1/4/16/32	80,000	0	80,000	0	80,000	0
shell, random + gaps/backpressure/malformed/reconfig	50,000	0	50,000	0	50,000	0
shell, Stim + gaps/backpressure	50,000	0	50,000	0	50,000	0

the boolean is materialised and then tested again. Our patch (`v8/patches/0001-*.patch`) adds an arm64 instruction `kArm64V128AnyTrueRaw` (`umaxp; fmov`) and matches a covered `Simd128TestOp::kV128AnyTrue` in `VisitWordCompareZero`. Branches and deopts become `umaxp; fmov; cbz/cbnz`, and selects use `tst` plus flags; values with other uses keep the materialising lowering. The CL adds three instruction-selector unittests and an mjsunit test that covers every single-bit input and the zero vector in `br_if`, inverted `br_if`, `if`, `select` and multi-use shapes, before and after tier-up. Results are in §7.6.

7 Evaluation

Setup. Host: Apple M5 Pro, 18 cores, 48 GB RAM, Darwin 27.0.0. All toolchain versions are pinned in `ENVIRONMENT.md`. The machine was shared with an unrelated workload (BLOCKERS B6), so timing experiments were run last, and before/after comparisons interleave their runs.

7.1 Software baselines: accuracy

Figure 1 shows P_L per d-round block for PyMatching and both QSX-UF variants over $p \in [0.001, 0.01]$ and $d \in \{3, 5, 7, 9\}$. Each point runs until a target number of failures or a shot cap is reached, and carries an exact Clopper–Pearson 95% interval [Monte Carlo, Stim]. At $p = 0.003$, weighted QSX-UF has $P_L = 1.92 \times 10^{-3}$ against 1.51×10^{-3} for PyMatching at $d = 7$ (ratio 1.28), and 7.93×10^{-4} against 5.58×10^{-4} at $d = 9$ (ratio 1.42). The $d = 7/9$ curve crossings are at $p \approx 0.0072$ (QSX-UF) and 0.0078 (PyMatching). The 3-bit weights are essential: the unweighted variant is markedly worse at large d (Fig. 1, right panel). The accuracy gap is the price of the order-independent, hardware-friendly algorithm, and it grows with distance.

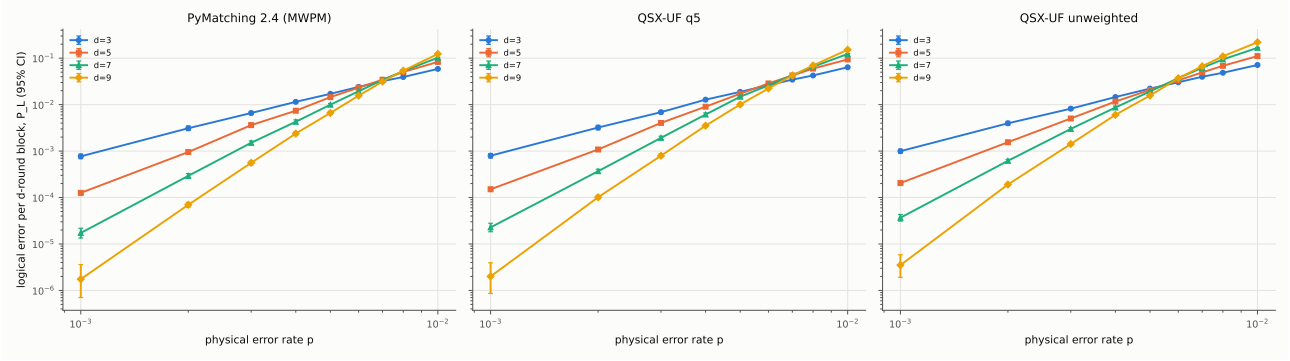


Figure 1: P_L vs. physical error rate (95% CIs), circuit-level depolarizing noise, d rounds.

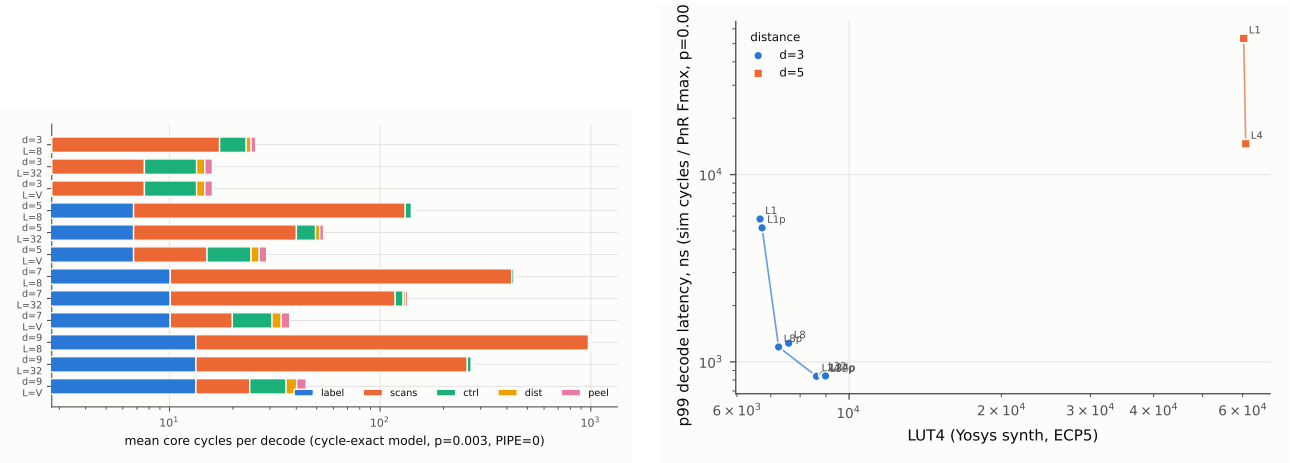


Figure 2: Left: mean core cycles per decode by FSM phase [sim]. Right: LUT4 [synth] vs. p99 latency [sim cycles / PnR Fmax] for every configuration that places and routes; labels give LANES and p=PIPE.

7.2 RTL correctness: co-simulation

Table 2 summarises the campaign on the final RTL: 3,840,000 syndromes, with 0 mismatches in status, round count, observable, full correction bitmap or cycle count. The shell runs inject malformed frames and incomplete configuration sessions. Every malformed frame was answered with `FRAME_ERROR` and every frame after an incomplete session with `CFG_INVALID`, in order, as the model predicts.

7.3 Hardware: cycles, resources and latency

Where the cycles go. The cycle-exact model splits every decode into label propagation, the LANES-wide scans (cluster parity/boundary and the activity broadcast), FSM control, BFS distances and peeling. An identity on every shot checks the split exactly (Fig. 2). At LANES= 8 the scans take 86% of all cycles at $d = 5$ and 94% at $d = 7$. With one lane per vertex (LANES= V) they drop to 26% at $d = 7$, and the mean cost falls from 439 to 37 cycles. The compute-unit parameter is therefore the main latency lever of this microarchitecture [sim].

Resources and timing. Table 3 lists every configuration. The fully parallel edge and vertex fabrics make area grow roughly with the number of graph edges. At $d = 3$ every configuration fits comfortably and closes at up to 64.8 MHz. With PIPE= 1, a register in the label path, Fmax rises by 18% (LANES= 32) for a few extra cycles. At $d = 5$ only LANES$\leq$ 4 places, at 99% logic utilisation and 32.6 MHz. Wider scans fail placement, and every $d \geq 7$ configuration exceeds the device after synthesis alone. The clock period is dominated by routing, not logic. At $d = 3$ (LANES= 8) the critical path has 12.5 ns of routing against 4.5 ns of logic. At $d = 5$, near

Table 3: FPGA implementation on the Lattice LFE5U-85F (open flow). LUT4/FF after Yosys synthesis [synth]; logic utilisation and Fmax after nextpnr place-and-route [PnR]; latency per d-round block at $p = 0.003$ including the shell, cycles from the cycle-exact model divided by Fmax [sim/PnR]; fallback = share of frames that would exceed the $d \times 1\,\mu$s block deadline and be answered WATCHDOG (then decoded in software).

configuration	LUT4	FF	util. %	Fmax MHz	p99 ns	max ns	fallback %
$d = 3$, q5, L1, P0	6,666	1,375	11	55.8	5,792	5,917	40.40
$d = 3$, q5, L1, P1	6,727	1,455	11	63.5	5,198	5,309	39.05
$d = 3$, q5, L8, P0	7,593	1,372	12	56.4	1,258	1,382	0.00
$d = 3$, q5, L8, P1	7,254	1,452	12	64.9	1,202	1,309	0.00
$d = 3$, q5, L32, P0	8,883	1,371	14	55.0	854	982	0.00
$d = 3$, q5, L32, P1	8,947	1,451	14	64.8	833	941	0.00
$d = 5$, q5, L1, P0	60,275	7,753	98	32.6	53,333	76,176	100.00
$d = 5$, q5, L4, P0	60,840	7,751	99	32.6	14,610	20,859	92.51
$d = 5$, q5, L8, P0	63,211	7,750	PnR: no legal placement (device full)				
$d = 5$, q5, L8, P1	62,722	8,254	PnR: no legal placement (device full)				
$d = 5$, unweighted, L8, P0	57,993	6,746	95	32.1	–	–	–
$d = 5$, q5, L16, P0	70,546	7,749	PnR: no legal placement (device full)				
$d = 5$, q5, L16, P1	67,160	8,253	PnR: no legal placement (device full)				
$d = 5$, q5, L32, P0	78,316	7,748	PnR: no legal placement (device full)				
$d = 7$, q5, L4, P0	262,677	24,258	exceeds the device after synthesis				
$d = 7$, q5, L8, P0	242,409	24,257	exceeds the device after synthesis				
$d = 7$, q5, L8, P1	239,897	25,817	exceeds the device after synthesis				
$d = 7$, q5, L16, P0	–	–	not finished				
$d = 7$, q5, L16, P1	–	–	not finished				
$d = 9$, q5, L8, P0	–	–	not finished				
$d = 9$, q5, L8, P1	–	–	not finished				
$d = 3$, q5, L24, P0	8,620	1,371	13	56.1	838	963	0.00
$d = 3$, q5, L24, P1	8,981	1,451	14	64.2	842	951	0.00
$d = 5$, q5, L120, P0	115,386	7,747	exceeds the device after synthesis				
$d = 5$, q5, L120, P1	109,311	8,251	exceeds the device after synthesis				

full utilisation, routing grows to 24.1 ns, which is why Fmax drops (`results/M2/fpga/critical_paths.json`).

Latency vs. the CPU. Figure 3 compares the best placed configuration per distance with single-thread CPU decoding of the same shots. At $d = 3$ the RTL p99 is 833 ns (d3_q5_L32_PIPE1), against 0.54 µs for PyMatching (RTL/CPU ratio 1.54). At $d = 5$ it is 14,610 ns against 3.12 µs (ratio 4.68). On this device the hardware decoder is *not* faster in typical or p99 latency. Its advantage lies in the tail. The worst case observed in hardware is 941 ns at $d = 3$, against 4.75 µs for the CPU, which is 5.0× lower. More importantly, the hardware bound is guaranteed rather than observed. Property C2 proves that a decode never exceeds its `cycle_budget`, and the shell proofs guarantee that a frame over budget comes back as a `WATCHDOG` status, never as a wrong valid result. Against a block deadline of $d \times 1$ µs (an assumed round time), with the budget set to fit, the hardware finishes 100.0% of $d = 3$ frames itself. At $d = 5$ it finishes only 7.5% of them; the rest go to the verified software path.

Activity. The toggle-count proxy grows from 1,904 bit toggles per decode at $d = 3$ to 34,088 at $d = 5$ and 269,690 at $d = 7$ ($p = 0.003$) [estimate]. It is a relative estimate of dynamic energy, not a power number, and it cannot be compared with the CPU.

7.4 The objective

We evaluate the project objective $J = \alpha\, \mathrm{P_L} + \beta\, T + \gamma\, A$ on every measured configuration, each metric normalised by its maximum (`bench/objective.py`). The energy term is omitted because CPU energy was not measured. Under T = p99 latency, no RTL configuration minimises J for any weighting at $d = 3$ (0% of the weight simplex) or at $d = 5$ (0%): a CPU core running PyMatching is better or equal in every metric. Under T = worst-case

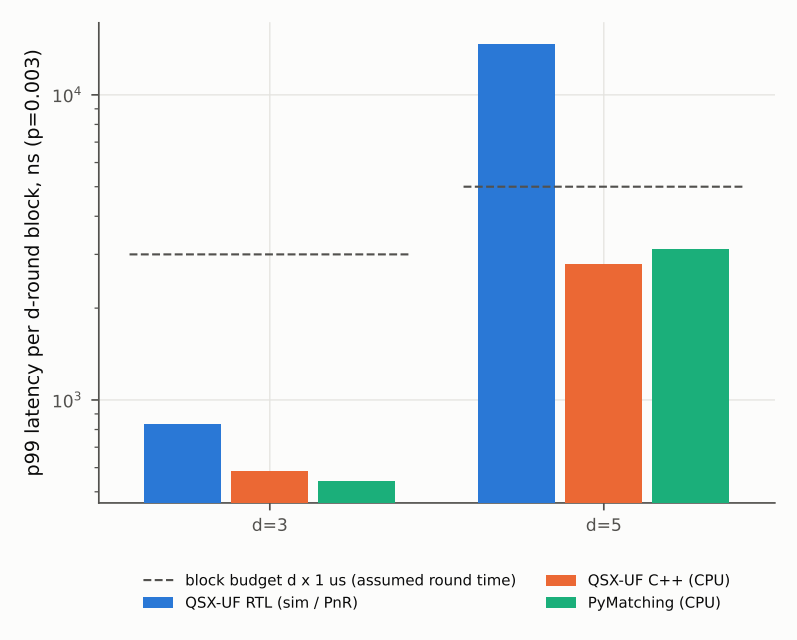


Figure 3: p99 latency per d-round block at $p = 0.003$: best placed RTL configuration [sim cycles / PnR Fmax] vs. QSX-UF and PyMatching on one CPU core [CPU]. Dashed: the $d \times 1\,\mu s$ block budget (assumed round time).

Table 4: Core vs. verified shell (Yosys `synth_ecp5 -noflatten`, LANES=8) and the shell's framing/handshake cycles per frame (Verilator, one frame in flight).

	core			shell				
	LUT4	FF	BRAM	LUT4	FF	BRAM	shell LUT	cycles
$d = 3$	4,939	866	0	305	578	1	5.8%	10.0
$d = 5$	39,032	5,380	0	1,149	2,434	1	2.9%	16.0

latency, which for the RTL is bounded by proof and for the CPU only observed, 4 configurations form the $d = 3$ Pareto set. RTL configurations minimise J for 23% of the weightings at $d = 3$ and 6% at $d = 5$, namely those that weight the deadline heavily. Co-design pays off exactly when the worst case is what matters.

7.5 Cost of the verified mechanisms

Table 4 separates the decoder core from the verified shell with a hierarchical synthesis [synth]. The shell contains the commit-on-last frame FIFO (backpressure, I1/P3), the atomic configuration session with its written-word bitmap and range check (P5/P6), and the dispatcher, round-tag path and status gating (P1–P4). At $d = 5$ it costs 1,149 LUT4 against 39,032 for the core (2.9%), but 31.1% of the flip-flops. Most of those FFs hold the threshold table, which any configurable decoder needs; the atomicity bookkeeping itself is one bit per configuration word plus one range-error bit. In time, framing and the handshakes add 10.0, 16.0 and 30.0 cycles per frame at $d = 3, 5, 7$ [sim]: the input header and syndrome beats, the dispatcher's load and the output header. Formal-only logic sits under `ifdef FORMAL` and is not synthesised.

7.6 WebAssembly runtime and the V8 patch

Portability and bit-exactness. Every native, Node.js, d8 and headless-Chrome build of the decoder produces identical hashes over statuses, observables and correction lists on the shared test vectors (all distances match). Figure 4 compares steady-state decode time. At $d = 5$ the Wasm SIMD build in d8 takes 1.22× the time of native clang `-O3` with NEON [CPU]. Headless Chrome's `performance.now()` resolution is coarse even when cross-origin isolated (BLOCKERS B7), so Chrome is reported by throughput only.

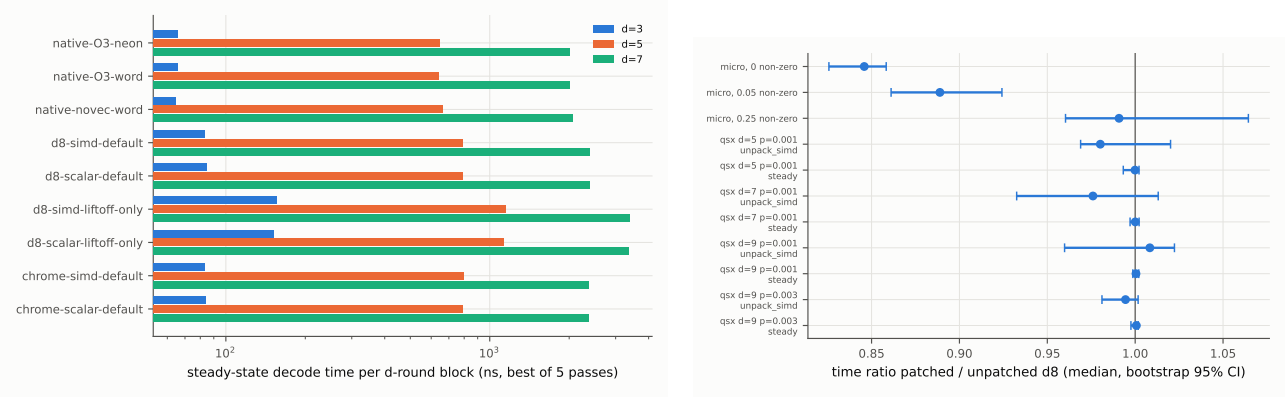


Figure 4: Left: steady-state decode time per d-round block, native vs. Wasm (d8 tiers, Chrome) [CPU]. Right: V8 patch, patched/unpatched time ratio with bootstrap 95% CIs [d8 measurement].

Table 5: Steady-state decode time per d-round block (ns, best of 5 passes, $p = 0.003$) and Wasm compile time at $d = 5$ (ms). All builds are bit-exact.

build / engine	$d = 3$	$d = 5$	$d = 7$	compile
`native-O3-neon`	66	648	2,015	–
`native-novec-word`	65	663	2,065	–
`node-simd`	93	870	2,557	0.4
`node-scalar`	94	832	2,577	0.3
`d8-simd-default`	83	789	2,386	0.3
`d8-scalar-default`	84	788	2,392	0.3
`d8-simd-liftoff-only`	156	1,146	3,382	0.3
`d8-simd-turbofan-only`	83	774	2,321	0.2
`chrome-simd-default`	83	799	2,371	0.4
`chrome-scalar-default`	84	787	2,380	0.4

The V8 patch. In the real decoder module, all 3 `v128.any_true` sites change from `umaxp; fmov; cmp; cset; cbnz` to `umaxp; fmov; cbz/cbnz` (3 of 3 fused). The patch applies cleanly to the recorded upstream revision. 2,955 V8 tests pass on the patched build (0 failures), covering the arm64 instruction-selector and scheduler unittests, the Wasm SIMD execution unittests, and `mjsunit/wasm`. The new mjsunit test passes on both builds and fails on a deliberately broken lowering that moves only the low half of the 64-bit reduction. In interleaved A/B runs of the two d8 binaries (medians, bootstrap 95% CIs) [d8 measurement], the zero-block scan takes 0.85× the unpatched time on all-zero data (CI 0.83–0.86) and 0.89× when a small share of blocks is non-zero. The gain disappears once the branch is mostly taken (0.99×, CI 0.96–1.06). In the decoder itself the effect is not measurable: the SIMD unpack kernel at $d = 9$, $p = 0.001$ runs at 1.01× (CI 0.96–1.02), and full decode time at 1.00×. Its 16-byte blocks are seldom all zero at these sizes, and union-find dominates the decode. The patch is a verified code-generation improvement for sparse-scan kernels, not a decoder speed-up.

7.7 Extensions

Learned decoding. A small message-passing GNN over the decoding graph, trained on Stim data at several p, reaches $P_L = 9.11 \times 10^{-3}$ at $d = 5$, $p = 0.003$, against 3.83×10^{-3} for QSX-UF and 3.19×10^{-3} for PyMatching, i.e. 2.85× PyMatching. At $d = 3$ the ratio is 1.08. Its single-thread CPU inference takes 184 µs at batch 1 and 139.8 µs per shot batched [CPU]. That is far slower than either classical decoder. It is a baseline, not a competitor to large learned decoders [3].

Adaptive policy under noise shifts. An online policy estimates (p, m) from two window statistics (the detection-event density and the fraction of adjacent defect pairs joined by time-like edges). It then reconfigures the accelerator's threshold table through the verified atomic session (P5/P6), or routes windows to PyMatching

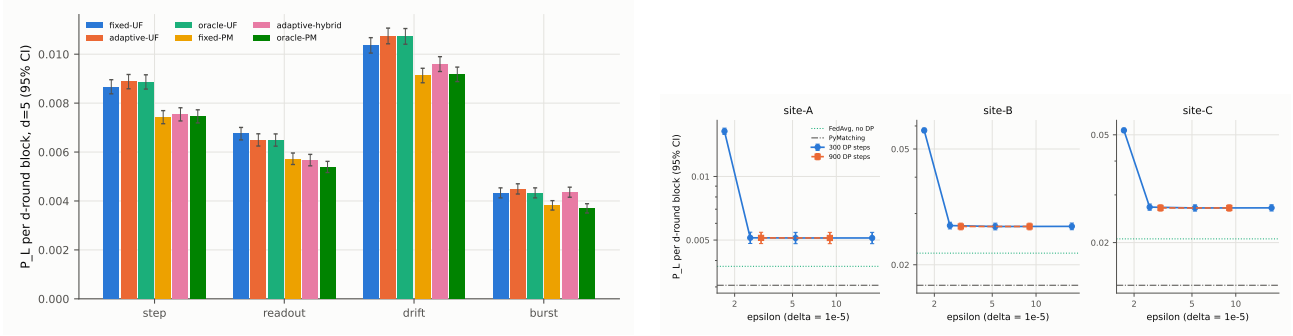


Figure 5: Left: P_L per strategy and noise-shift scenario ($d = 5$, 95% CIs). Right: federated DP-SGD, ε vs. P_L per site, with non-DP FedAvg and PyMatching for reference.

Table 6: Federated GNN decoder training, $d = 3$, three sites with different noise. P_L in %, per site's own test set; ε per site at $\delta = 10^{-5}$ (record level); upload = total masked-update bytes from all sites.

regime	ε	A	B	C	upload MB
PyMatching (true noise model)	–	0.31	1.70	1.39	–
local only (no sharing)	–	0.50	1.98	1.69	–
centralized (pooled data)	–	0.34	1.93	1.75	–
FedAvg + SecAgg, non-IID	–	0.38	2.19	2.06	5.8
FedAvg + SecAgg, IID split	–	0.34	1.95	1.82	5.8
FedAvg + DP-SGD, lot 512, 4000 steps, $\sigma = 0$	∞	0.35	2.01	1.88	5.8
FedAvg + DP-SGD, lot 512, 4000 steps, $\sigma = 0.6$	4.80	0.51	2.71	2.69	5.8
FedAvg + DP-SGD, lot 512, 4000 steps, $\sigma = 1$	1.10	0.51	2.71	2.69	5.8
FedAvg + DP-SGD, lot 512, 4000 steps, $\sigma = 2$	0.33	0.51	2.71	2.68	5.8
FedAvg + DP-SGD, lot 8192, 300 steps, $\sigma = 0$	∞	0.51	2.71	2.68	2.9
FedAvg + DP-SGD, lot 8192, 300 steps, $\sigma = 0.6$	17.62	0.51	2.71	2.69	2.9
FedAvg + DP-SGD, lot 8192, 300 steps, $\sigma = 1$	5.25	0.51	2.71	2.68	2.9
FedAvg + DP-SGD, lot 8192, 300 steps, $\sigma = 1.5$	2.56	0.51	2.73	2.70	2.9
FedAvg + DP-SGD, lot 8192, 300 steps, $\sigma = 2$	1.70	1.64	5.79	5.19	2.9
FedAvg + DP-SGD, lot 8192, 900 steps, $\sigma = 1$	9.03	0.51	2.71	2.68	8.7
FedAvg + DP-SGD, lot 8192, 900 steps, $\sigma = 2$	3.04	0.51	2.71	2.69	8.7

("hybrid"). We compare it with fixed decoders and with oracles that know the true noise (Fig. 5). After a step change in p, the hybrid policy matches the PyMatching oracle (7.54×10^{-3} vs. 7.45×10^{-3}) within 2,000 shots. Re-weighting QSX-UF alone did *not* help when p shifts: adaptive and oracle UF are at 8.87×10^{-3} and 8.86×10^{-3}, against 8.67×10^{-3} for fixed weights. The median-normalised 3-bit thresholds are nearly invariant under a uniform change of p. Re-weighting helped modestly when the *relative* error rates change, as in the readout-error shift (6.49×10^{-3} vs. 6.75×10^{-3}). The policy costs at most 256 ns per shot [CPU]; its statistics are counters that the shell could compute in hardware.

Federated training with differential privacy. Three simulated sites have different noise (Table 6). FedAvg with pairwise-mask secure aggregation approaches centralized training. The first DP-SGD configuration (lot 512) gave P_L that did not depend on σ. Ablations traced this to a plateau that DP-limited training converges to, rather than to clipping (DECISIONS D9/D10, `results/M7/dp_debug.json`). With large lots (8192), privacy from $\varepsilon = 17.6$ down to $\varepsilon = 2.56$ ($\delta = 10^{-5}$, RDP accountant) costs nothing beyond that plateau. At $\varepsilon = 1.70$ P_L rises at least 1.9× at every site. Tripling the step budget lets the noisiest run reach the plateau, but only at a higher ε. This is a single-process simulation; the masks assume honest-but-curious parties and no dropouts.

Secure experiment infrastructure. Syndrome traces and result manifests are Ed25519-signed, results are stored with AES-256-GCM, and runs are started and stopped through an mTLS control API, all with the `cryptography` library and Python's `ssl`. 7 tests pass (0 failures), including the negative cases: a tampered trace, a manifest with a wrong key, an unsigned or replayed command, and a missing or untrusted client

certificate are all rejected. Verification runs at 2,604 MB/s and encryption at 3,961 MB/s. A fresh mTLS connection costs 1.19 ms (p50), against 0.18 ms for plain HTTP [CPU].

End to end. The integration demo (`bench/e2e_demo.py`) samples 5,000 Stim shots and streams them through the RTL accelerator in Verilator. The accelerator is configured by an atomic session with a small growth-round limit, and 0 frames mismatch the model. The host implements `QSXProtocol.tla` and asserts its safety invariants at run time. 4,831 rounds were committed from hardware and 169 from the software fallback, covering 119 hardware timeouts and 50 injected lost responses. The committed predictions are signed and stored encrypted. A browser view (M9, `viz/`) verifies the signature with WebCrypto and replays the trace through the Wasm decoder, reproducing the committed predictions (yes).

8 Threats to validity and limitations

- **No board.** Latency is cycles from a model that is cycle-exact with the RTL, divided by the post-PnR Fmax of one seed. Host–FPGA transport, clock-domain crossings and on-board jitter are not included. Energy is a toggle-count proxy, not Watts.
- **Proof scope.** k-induction of the real core is for $d = 3$ only. Larger cores are covered by co-simulation and one-step Z3 equivalence, which is not a proof of end-to-end equivalence. The TLA+ model is checked for small constants (stated in Table 1). No proof says anything about $\mathrm{P_L}$.
- **Noise model.** Uniform circuit-level depolarizing noise, memory experiments only, decoder graph built for the true p (except in the noise-shift study). Real devices have leakage, crosstalk and non-uniform rates.
- **CPU baselines.** One thread, batch-1 latency, timer resolution 41 ns. The CPU numbers exclude syndrome acquisition, just as the hardware numbers exclude transport.
- **Federated study.** A single-process simulation. The secure-aggregation masks assume no dropouts and honest-but-curious parties. The DP guarantee is per site, at record level, for the stated (ε, δ).

9 Related work

Union-find decoding was introduced by Delfosse and Nickerson [8] and extended to weighted graphs [15]. AFS [6] proposed a UF-based hardware decoder architecture. Helios [19] implemented a distributed UF decoder on FPGA, and Barber et al. [2] reported a real-time collision-clustering decoder on FPGA and ASIC. Software MWPM via sparse blossom [14] in PyMatching [13] is our baseline. Neural decoders range from GNNs [18] to transformer-based models trained on hardware data [3]. Our GNN is a small baseline, not a competitor to these. On the systems side, we use TLA+ [17], Yosys/nextpnr [22] and Z3 [7]. We use FedAvg [20], DP-SGD [1] with Rényi accounting [21, 23], pairwise-mask secure aggregation [5] and Ed25519 [4]. What we believe is new is the combination: a UF variant designed for bit-exact co-simulation, with its shell and host protocol verified to stated bounds, and evaluated end to end with open tools.

10 Conclusion

For the question we set out to answer, the evidence supports a mixed result. A union-find decoder designed for bit-exact co-simulation and wrapped in a formally verified shell delivers what the verification promises: a latency bound that holds by proof, with a verified escape hatch to software. On a low-cost FPGA it does not deliver lower typical latency than one modern CPU core. Its accuracy trails matching by a factor that grows with distance, and full parallelism exhausts the device at $d = 5$. The cycle accounting points to the next design: the serial scans, not the union-find fabric, dominate the cycle count. A time-multiplexed fabric with wide scans, on a faster FPGA and measured on a board, is where the co-design question should be asked again. The verification, the artefacts and the reproduction scripts carry over unchanged.

References

[1] Martín Abadi, Andy Chu, Ian Goodfellow, H. Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. Deep learning with differential privacy. In *ACM Conference on Computer and Communications Security (CCS)*, 2016.

[2] Ben Barber et al. A real-time, scalable, fast and highly resource efficient decoder for a quantum computer. arXiv:2309.05558, 2023.

[3] Johannes Bausch et al. Learning high-accuracy error decoding for quantum processors. *Nature*, 635:834–840, 2024.

[4] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2:77–89, 2012.

[5] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. Practical secure aggregation for privacy-preserving machine learning. In *ACM Conference on Computer and Communications Security (CCS)*, 2017.

[6] Poulami Das, Christopher A. Pattison, Srilatha Manne, Douglas M. Carmean, Krysta M. Svore, Moinuddin Qureshi, and Nicolas Delfosse. AFS: Accurate, fast, and scalable error-decoding for fault-tolerant quantum computers. In *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2022.

[7] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2008.

[8] Nicolas Delfosse and Naomi H. Nickerson. Almost-linear time decoding algorithm for topological codes. *Quantum*, 5:595, 2021.

[9] Austin G. Fowler, Matteo Mariantoni, John M. Martinis, and Andrew N. Cleland. Surface codes: Towards practical large-scale quantum computation. *Physical Review A*, 86:032324, 2012.

[10] Craig Gidney. Stim: a fast stabilizer circuit simulator. *Quantum*, 5:497, 2021.

[11] Google Quantum AI and Collaborators. Quantum error correction below the surface code threshold. *Nature*, 638:920–926, 2025.

[12] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and J. F. Bastien. Bringing the web up to speed with WebAssembly. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2017.

[13] Oscar Higgott. PyMatching: A Python package for decoding quantum codes with minimum-weight perfect matching. *ACM Transactions on Quantum Computing*, 3(3), 2022.

[14] Oscar Higgott and Craig Gidney. Sparse blossom: correcting a million errors per core second with minimum-weight matching. arXiv:2303.15933, 2023.

[15] Shilin Huang, Michael Newman, and Kenneth R. Brown. Fault-tolerant weighted union-find decoding on the toric code. *Physical Review A*, 102:012419, 2020.

[16] Abhinav Jangda, Bobby Powers, Emery D. Berger, and Arjun Guha. Not so fast: Analyzing the performance of WebAssembly vs. native code. In *USENIX Annual Technical Conference (ATC)*, 2019.

[17] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[18] Moritz Lange, Pontus Havström, Basudha Srivastava, Valdemar Bergentall, Karl Hammar, Olivia Heuts, Evert van Nieuwenburg, and Mats Granath. Data-driven decoding of quantum error correcting codes using graph neural networks. arXiv:2307.01241, 2023.

[19] Namitha Liyanage, Yue Wu, Alexander Deters, and Lin Zhong. Scalable quantum error correction for surface codes using FPGA. arXiv:2301.08419, 2023.

[20] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017.

[21] Ilya Mironov. Rényi differential privacy. In *IEEE Computer Security Foundations Symposium (CSF)*, 2017.

[22] David Shah, Eddie Hung, Clifford Wolf, Serge Bazanski, Dan Gisselquist, and Miodrag Milanovic. Yosys+nextpnr: An open source framework from Verilog to bitstream for commercial FPGAs. In *IEEE Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2019.

[23] Ashkan Yousefpour et al. Opacus: User-friendly differential privacy library in PyTorch. arXiv:2109.12298, 2021.